

Ray-dispatch launch-model measurements on Arc Pro B70 (Xe2): findings and questions

Version 1.2 — 2026-08-04

Changelog

- **1.2** (2026-08-04): finding 1's small-scene "no tax" control retired (superseded by tree evolution) — the footprint claim now rests on a thin-vs-fat raygen A/B; added the mode-2 build-variance caveat (ratios are bands, not points, drivers labeled per sample); corrected the wave-atomics observation after an A/B-lever repair reversed its verdict (neutral -> small cross-vendor regression); updated the Intel-defaults note (compute-vs-DXR advantage re-verified under camera motion; the renderer's DXR pipeline now defaults to the zero-TraceRay mode on Intel adapters); fixed the compute-arm and work-graph repro commands; withdrew the historical 0.87-0.93 marginal ratios; added the compute-door register-cliff datum to hypothesis 2; Q4's many-record-SBT counterfactual is now under construction (with the NVIDIA alias-dedupe datum), and the thin/fat A/B is on master.
- **1.1** (2026-08-04): August re-captures on driver 8805 added to finding 1; work-graph AV re-confirmed on 8805.
- **1.0** (2026-08): initial draft (July rows, driver 8515).

We develop an open-source hybrid renderer (frustracer, <https://github.com/fechols/frustracer>) that can run the *same* ray-traced frame through several dispatch models — a compute kernel with inline `RayQuery`, and a DXR `DispatchRays` pipeline at three inlining levels — from one shader source, with correctness gates locking the arms to one image (exact-zero counters where the hardware permits bit identity; tight statistical class/radiance bounds across the watertight-vs-Moller-Trumbore intersector difference), so a fast arm cannot be fast by computing less. That makes it an unusually clean instrument for pricing the dispatch models against each other, and on the B70 it produced one result we cannot explain from public documentation, two results your RT developer guide predicts qualitatively (we can now attach magnitudes), and one crash that looks like a driver bug. Numbers, method, and questions below.

Environment of record

GPU	Intel Arc Pro B70 (Battlemage / Xe2), reported caps: RT tier 1.1, SM 6.8, wave lanes 16..32, WorkGraphsTier 1.0
Drivers	32.0.101. 8805 (all August rows below); July rows were taken on 32.0.101.8515 or earlier and are marked
OS	Windows 11 Pro 10.0.26200
Control GPU	RTX 4090 in the same box (adapter selection is explicit and verified by name per run)
Timing	D3D12 timestamp queries around per-pass markers, 120-frame windows, warm shader cache (we characterize and wait out the async recompile), deterministic camera path, 2 interleaved reps on the B70 (repeatability ± 0.002 ms)

Method: what is held constant

All arms share one HLSL source tree (the shading and intersection code is pasted into each compile unit), one acceleration structure, one camera path, one sample sequence. The arms:

- **compute** — `cs_6_5` kernel, primary + all secondary rays as inline `RayQuery`;

- **mode 2** — `lib_6_5 raygen, DispatchRays` as a bare launch grid, `zero TraceRay` anywhere (all rays inline `RayQuery` inside the raygen);
- **mode 1** — primary `TraceRay` -> closest-hit, all secondaries inline `RayQuery` inside the hit shader (`MaxTraceRecursionDepth 1`);
- **mode 0** — the by-the-book all- `TraceRay` pipeline (identifier-only SBT records, one hit group for shading — a single-record SBT, which matters below).

Cross-arm image equality is gated (exact-zero counters where the hardware permits bit identity), so a fast arm cannot be fast by computing less.

Finding 1 — the one we can't explain: DispatchRays costs ~2x on register-fat shaders with zero TraceRay

On our large scene (34.4M triangles; the shaders carry alpha-cutout + tinted-transmission candidate loops, 100-byte material records, large bindless texture tables — i.e. register-fat), the identical full-screen traversal, same day, same driver (8805), same parked pose:

arm	trace time (ms)	ratio
compute (inline RayQuery)	1.28	1.0x
mode 2 raygen (zero TraceRay)	2.59	2.0x
mode 1 (one TraceRay + CH per pixel)	3.13	2.4x

Mode 2 contains no ray-dispatch, no sorting, no payload traffic — it is the compute loop launched through `DispatchRays` — and it still pays 2.0x.

That the tax scales with shader FOOTPRINT and not with dispatch count is pinned by a thin/fat A/B on the same pipeline (now on master as `--dxr-inline 3`; raw logs available): a THIN raygen — bare-hit primary `TraceRay` writing a 20 B record, no shading — costs 0.23-0.35 ms full-screen on our spin scenes and 0.54 ms on the large scene, through the same `DispatchRays` against the same acceleration structure that costs 2.9 ms with the fat shader resident. Complementary ablations point the same way: compiling out all secondary-ray TRAVERSAL from mode 1 (rng draws and control flow kept) collapses it 2.40 -> 0.48 ms — BELOW the full compute kernel — while removing the ray categories ONE AT A TIME sums to only 0.76 of that 1.92, and removing the glass chain alone "saves" 0.28 ms on a scene with zero transmissive geometry. Sub-additive savings from code that never executes are an occupancy/register signature, not ray work.

Two honesty caveats on the magnitudes. (1) Mode-2 numbers are build-unstable on Arc: on one day on 8805, a comment-level rebuild (source diffs in comments only) moved the procedural scene's mode 2 from 2.46 to 1.77 ms — each internally repeatable to ± 0.002 — and the July capture of the same arm and scene (8515, earlier tree) sat at 1.41. So read every mode-2 ratio here as a band, not a point, and note the mode1-vs-mode2 ORDERING flips between builds (the " ~ 0.5 ms scheduler hop" delta sits inside this variance). (2) An earlier revision of this brief claimed the tax is absent on slim-shader small scenes; that control has not survived tree evolution (our compute kernel has since improved to 0.60 ms on the 80k scene while mode 2 reads 1.8-2.5 across builds there), which is why the thin/fat A/B above, not scene size, now carries the footprint claim.

What we ruled out by construction or measurement: ray dispatch itself (mode 2 has none; the thin-raygen row shows the launch path itself is cheap), the scene/AS (identical), shader source (identical), fixed launch overhead (the thin raygen's 0.23-0.54 ms IS the launch + traversal floor; separately, we measured `ExecuteIndirect` dispatch overhead at

~11 us across 8 back-to-back indirect dispatches on Xe2 — your EI-in-hardware claim checks out), cold compiles (warm cache, windowed timestamps), and our own traversal structure (this comparison doesn't involve it).

We attempted to answer the obvious next question ourselves — what SIMD width and spill counts does the raygen get vs the byte-identical `cs_6_5` twin — via `IGC_ShaderDumpEnable` / `IGC_DumpToCustomDir` (also `...EnableAll`, `...PidDisable`), which produced zero files from the D3D12 UMD on 8805.

Finding 2 — your guide predicts this one; here are the magnitudes

Your RT developer guide documents that the Thread Sorting Unit sorts by shader record; our SBT has effectively one record, so the recursive pipeline pays full repacking for zero coherence benefit. Measured (spin benchmark, 1080p, 1 spp, GPU frame span, July driver):

scene	mode 0	mode 1	mode 2
procedural 80k	9.05	2.35	1.41
stress (5000 objects)	5.30	1.64	1.22
San Miguel low-poly 5.6M	6.75	1.94	1.29

Secondary- `TraceRay` dispatch was 68-84% of the whole pipeline's cost — and, for calibration, the same proportion holds on the RTX 4090 at its own scale (1.34 -> 0.26 on the procedural scene), so this is not an Arc-only effect; Arc's multiplier is ~1.4-1.5x NVIDIA's. (The mode-2 column above is the July/8515 capture; per the build-variance caveat in finding 1, mode-2 points have since ranged 1.8-2.5 on the procedural scene across builds on 8805 — mode 0 and mode 1 reproduce far more tightly.) The counterfactual your hardware is designed for — a genuinely many-record, material-sorted SBT where the TSU could earn its keep — is now UNDER CONSTRUCTION in the repo (`--dxr-sbt`, a class-partitioned SBT ladder; our shading was deliberately one uber-path until now). First datum, possibly useful to you: at the ladder's alias rung (8 renamed exports of ONE closest-hit — identical code, distinct sort keys) the NVIDIA driver dedupes all 8 records to a single shader identifier while the B70 mints 8 distinct ones, so pure record-sort effects are measurable on Arc only; results to follow at the specialized-record rung. SER remains out of reach at the reported SM 6.8.

Finding 3 — per-sample occupancy signature

At higher samples-per-pixel, mode 1's *marginal* cost per extra sample on the B70 is 2.2 ms/sample vs mode 2's 1.11 — the candidate-loop-fattened closest-hit shader appears to pay occupancy per sample where the fused raygen pays it once. Consistent with finding 1's "occupancy under the RT launch regime" shape.

Bug report — DispatchGraph access violation (work graphs)

- Device reports `WorkGraphsTier 1.0`; the state object builds without complaint; backing memory ask is 517.62 MB (min == max == 542769152, granularity 1).
- First `DispatchGraph` takes an access violation (process exit 0xC0000005) with the debug layer AND GPU-based validation both silent.
- Reproduced identically on 32.0.101.8515 and 32.0.101.8805.
- The identical graph runs on the RTX 4090 and produces results bit-identical to our `ExecuteIndirect` ladder (same-seed image diff exactly 0.0), with an 82 MB backing ask — so we believe the graph itself is well-formed. Repro is one env var on our renderer (`FR_WORKGRAPH=1`); happy to provide a standalone capture if useful.

Corroborating observations (things that match your guidance)

- Our ray-shooting kernels carry zero groupshared memory (per your LDS-shares-L1-with-the-RTU note); the level-recursion kernels that do carry LDS shoot no rays.
- Replacing queue atomics with wave-intrinsic aggregation is a small cross-vendor REGRESSION in our workload — with a correction to our own method: an earlier revision said "neutral", but that A/B's disable lever turned out to reach only one of the two aggregation sites. The repaired A/B prices the wave choreography at ~+3% frame span / ~+10% on the affected pass on the B70 (a smaller but real cost on the RTX 4090 too), so we are reverting to plain atomics. Our queue atomics were simply never contended enough to pay for the reduction — a data point for the limits of that guidance, not a complaint.
- Inline `RayQuery` throughput on the B70 is competitive: our compute-hosted tracers beat every DXR arm on Arc at the shipping 1-spp config on every scene we run — re-verified on the large scene parked AND under camera motion (compute-hosted frame span 3.3-3.5 ms vs mode-1 DXR's 5.4, same TLAS, both paying the same ~1.4 ms per-frame animation refit) — while the same comparison favors the DXR pipeline on Ada; the hardware balance genuinely inverts by vendor, which is why the renderer's Intel defaults differ. Consistent with findings 1-3, the renderer now also defaults its DXR pipeline to the zero-TraceRay mode 2 on Intel adapters specifically (decided on the large-scene span, 4.77 vs 5.36 ms, and finding 3's per-sample marginals — the small-scene points carry the build-variance caveat), while NVIDIA keeps mode 1: the preference crosses one, not just narrows. (An earlier revision quoted per-sample marginal ratios of 0.87-0.93 here; those were measured on a since-retired dispatch configuration and we no longer stand behind the specific figures.)

Hypotheses — explicitly guesses, ranked

1. The raygen runs under the bindless-thread-dispatch launch regime rather than as a plain compute walker: narrower effective SIMD width and/or per-thread RT-stack residency (`RTDispatchGlobals`) and a different occupancy calculus for high-GRF kernels. Your patents say BTD supports SIMD16 and SIMD32 — we don't know which our raygen gets.
2. Compiler path: `lib_6_5` DXIL library vs `cs_6_5` through different IGC entrances — different register allocation / width policy for the same code. One datum complicates a pure two-doors story: a deferred-shading `cs_6_5` kernel of ours (record read + the same fat shading, strictly LESS work than the compute arm, which also traverses the primary) runs ~1.9x slower than that compute arm (1.12 vs 0.60 ms) — so a knife-edge register-allocation cliff exists on the compute door too; the RT-stage door just seems to sit past it far more often (dead-code probes that neutralize never-executed feature arms move our compute-hosted kernels only ~2-3%, while mode 1's never-executed glass chain was worth 12% of its frame).
3. Driver per-dispatch overhead — largely ruled out (the tax scales with per-pixel work, not dispatch count).

Questions

1. Is the fat-raygen-vs-compute ratio (finding 1) a known characteristic of the launch path on Xe2? Is there a driver or compiler lever (width, stack sizing, occupancy hints) that closes any of it — or is it silicon?
2. What SIMD width / GRF configuration does our mode-2 raygen actually compile to vs its `cs_6_5` twin — and is there a supported way for us to see that on shipping drivers (the IGC shader-dump env route appears inert on 8805)?
3. Is the DispatchGraph AV a known issue? We can hand you a minimal repro.
4. For workloads shaped like ours — thin SBT, fat uber-shader — would you expect a many-hit-group TSU-sorted pipeline to beat inline-in-compute? We have begun building that experiment (the `--dxr-sbt` ladder above)

and would gladly compare notes; if the answer is no, the developer guide may be worth a paragraph saying so, because today it optimizes within the pipeline and never prices the pipeline against compute.

5. Anything in the above you'd expect to change materially on a near-term driver? We stamp every number with a driver version and re-run on updates; happy to be a regression canary.

Repro

Everything is driven by CLI flags / env levers on the renderer (Windows, D3D12):

```
cargo run --release -- --spin path --gpu --spin-plain --gpu-timing --prefer-intel    # compute arm
cargo run --release -- --spin path --dxr --dxr-inline 2 --gpu-timing --prefer-intel  # mode 2
cargo run --release -- --spin path --dxr --dxr-inline 1 --gpu-timing --prefer-intel  # mode 1
cargo run --release -- --spin path --dxr --dxr-inline 0 --gpu-timing --prefer-intel  # mode 0
```

(`--gpu` without `--spin-plain` runs our hybrid quadtree tracer, a different algorithm — the compute ARM in the tables above is the plain per-pixel reference kernel, `--spin-plain`.)

The work-graph AV: current builds detect the picked Intel adapter and REFUSE `FR_WORKGRAPH=1` with a loud line precisely because of this crash — reproducing it requires deleting that one-line refusal arm in `src/gpu/trace.rs` (`grep FR_WORKGRAPH`) or checking out a commit before it landed; we will gladly hand you the standalone capture instead.

The benchmark is deterministic (camera pose is a pure function of frame index), prints per-pass GPU-ms tables, and the correctness suites (`--check-gpu`, `--check-dxr`) gate the arms against each other bit-exactly where the hardware permits. The large-scene numbers use the renderer's composite world scene; we can provide exact poses, scripts, and raw logs.